AIMS RIC — Doctoral Training School 2026

ML-ESS: A Multi-Agent AI Research Assistant

Technical Report — April 2026

Jérémie N. Landu[1], **Samuler Kargbo**[1], **Cecilia Akyeampong**[1], **Abigail Nunoo**[1], and **Lambert**[1]

[1]African Institute for Mathematical Sciences — Research Innovation Centre (AIMS-RIC), Doctoral Training School 2026.
jeremy@aimsammi.org

Abstract

ML-ESS is a full-stack AI system that automates end-to-end research: given a natural language question, it orchestrates four specialised AI agents to search the web, synthesise evidence, draft a structured report, and evaluate its quality. Built as a *working demonstration prototype* for the AIMS-RIC Doctoral Training School 2026, the system exposes a REST API (FastAPI) and a Next.js web interface supporting real-time progress tracking and multi-provider LLM fallback. Every architectural decision prioritises clarity and modularity over production hardening, leaving well-defined extension points for future work.

Keywords: multi-agent systems, LLMs, automated research, FastAPI, Next.js, retrieval-augmented generation.

1 Introduction

Conducting research manually is time-consuming: it requires querying multiple sources, extracting claims, resolving contradictions, and drafting a coherent document. ML-ESS automates this workflow through a four-agent pipeline. Related work — AutoGPT [2], CrewAI [3], LangChain [4] — provides agent primitives but lacks a structured evidence chain with quality scoring. Lewis *et al.* [1] established RAG as the foundational pattern; ML-ESS extends it with typed intermediate artefacts and a self-evaluating stage.

The prototype runs end-to-end on a laptop with a single API key. It is designed as a *teaching platform*: each component is inspectable, each intermediate artefact is persisted, and adding a new research phase requires only inserting a new agent function into the pipeline.

Core objectives:

- Accept any research question in natural language.
- Autonomously gather and synthesise web evidence.
- Generate a structured Markdown report with citations.
- Score report quality on four dimensions (coverage, faithfulness, usefulness, hallucination rate).
- Deliver results via a clean web UI with real-time feedback.

2 System Architecture

The system comprises a Python/FastAPI backend and a Next.js frontend (Figure 1). The backend runs the four-agent pipeline in a background thread, persisting state in SQLite (WAL mode). The frontend polls the REST API every two seconds and renders the report when complete.

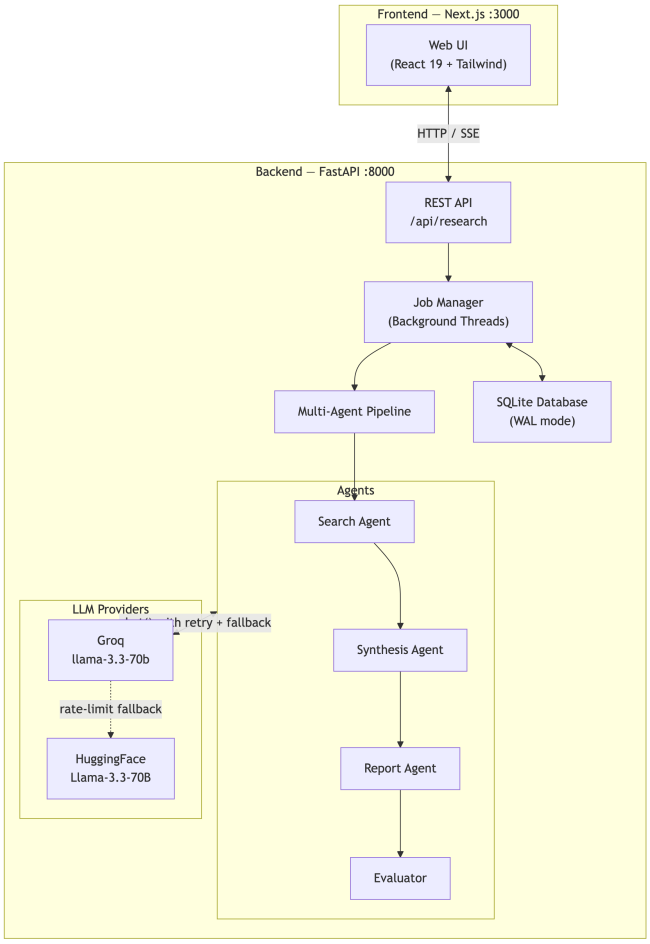

Figure 1: High-level system architecture.

Request lifecycle. A `POST /api/research` request returns a `job_id` immediately (HTTP 202). A background thread runs the pipeline and writes status updates to SQLite. The frontend polls `GET /api/research/{id}` every 2 s; an optional SSE stream at `.../events` pushes stage-change events in real time. Figure 2 shows the full sequence.

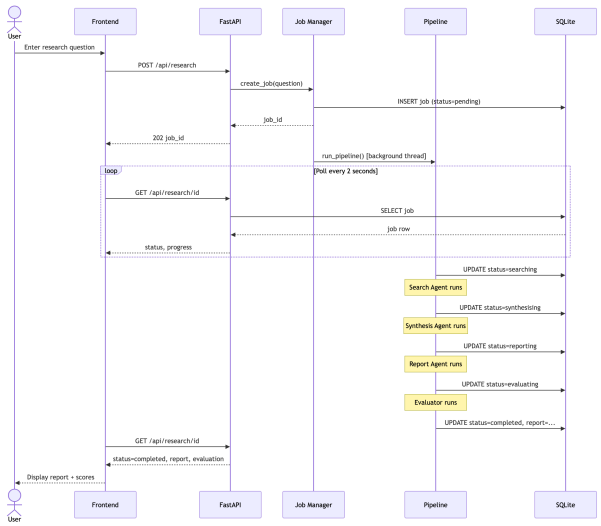

Figure 2: Request lifecycle sequence diagram.

Backend layout (`api/app/`): `agents/` — one file per agent; `api/` — FastAPI factory, routes, SSE, auth; `core/` — pipeline orchestrator, job manager, SQLite store, LLM client; `models/` — Pydantic request/response and shared-state schemas.

3 The Multi-Agent Pipeline

3.1 SharedState

All agents share a single `SharedState` Pydantic object that travels through the pipeline. It carries: `research_question`, `search_queries`, `sources` (title, URL, snippet, images, credibility notes), `evidence` (claim, quote, relevance score, source index), `themes` (name, summary, confidence: high/medium/low), `contradictions` (description, resolution), `report_outline`, `final_report`, and `evaluation` (the four scores). This typed contract makes each agent independently testable and the full pipeline trivially extensible.

3.2 Pipeline Flow

Each agent is a stateless function that reads from and writes to the shared `SharedState`. Figure 3 shows the four-stage flow.

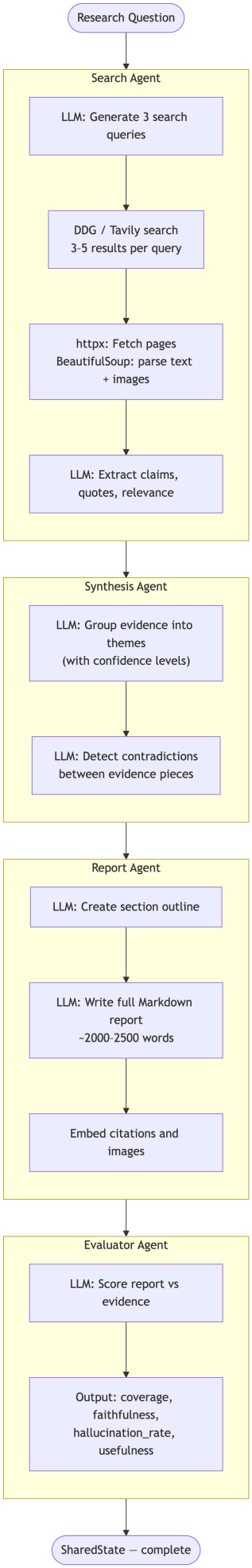

Figure 3: Four-stage agent pipeline.

1. **Search Agent** — generates 3 queries via LLM, fetches up to 15 pages (DuckDuckGo / Tavily), parses with BeautifulSoup4, extracts typed `Evidence` records.
2. **Synthesis Agent** — clusters evidence into `Theme` objects with confidence levels; detects `Contradiction` pairs.
3. **Report Agent** — builds a section outline then writes a ~2 000-word Markdown report with inline citations.
4. **Evaluator** — scores the report against the evidence on coverage, faithfulness, hallucination rate, and usefulness.

3.3 Evaluation Scores

The Evaluator outputs four scores in $[0, 1]$ (Table 1); lower is better for hallucination rate, higher is better for the rest.

Table 1: Example evaluation output.

Dimension	Score
Coverage	0.87
Faithfulness	0.92
Usefulness	0.85
Hallucination rate	0.08

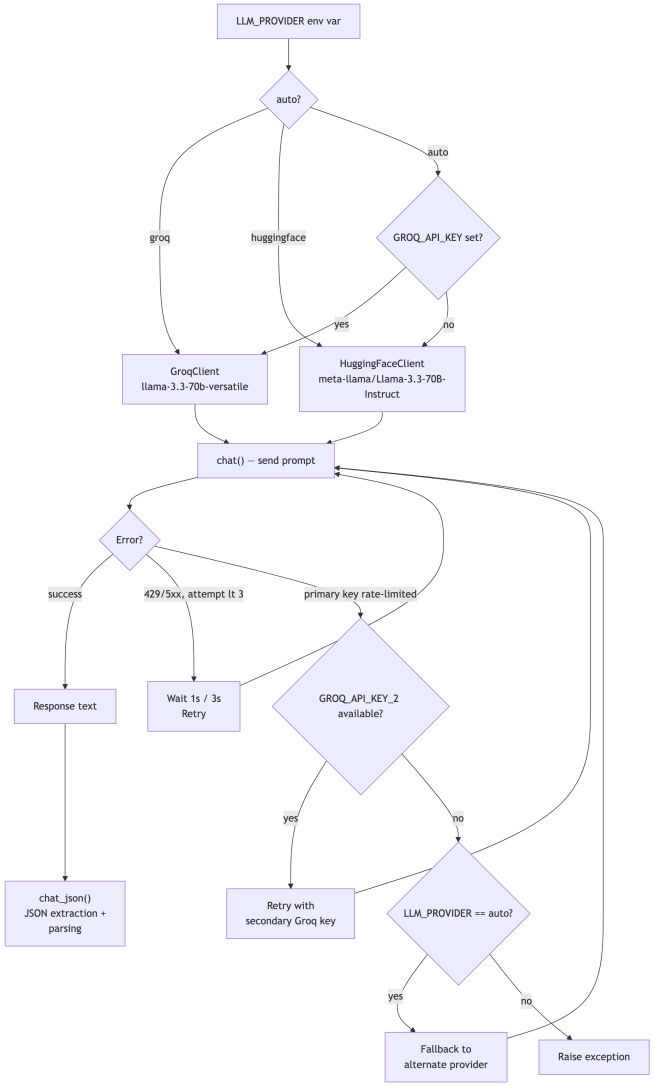

Figure 4: LLM retry and fallback strategy.

3.4 Search Agent

The Search Agent generates up to three LLM-crafted queries, fans them out to DuckDuckGo (or Tavily when `TAVILY_API_KEY` is set), de-duplicates up to 15 result URLs, fetches each page with `httpx` (10 s timeout), parses text and images with BeautifulSoup4, truncates to `MAX_CONTENT_LENGTH` characters, and finally calls the LLM to extract structured `Evidence` records (claim, quote, relevance score, source index).

3.5 LLM Provider Strategy

Figure 4 shows the retry and fallback logic. Groq (`llama-3.3-70b-versatile`) is the primary provider; HuggingFace Inference serves as fallback when Groq's rate limit is exhausted. Temperature is fixed at 0.3 for deterministic, reproducible outputs. A secondary Groq key (`GROQ_API_KEY_2`) can also be configured to extend rate-limit headroom before falling back to HuggingFace.

4 Data Persistence & API

4.1 Schema

The SQLite database has three tables. `JOBS` stores the primary record (job ID, question, status, serialised report, evaluation scores, source/evidence/ theme counts, timestamps, error). `STATES` holds the full serialised `SharedState` JSON for each job, enabling complete replay of any pipeline run. `EVENTS` is an append-only log of SSE events emitted during the pipeline (stage changes, queries ready, sources ready, job completed), enabling the frontend to reconstruct the live event stream on reconnect.

4.2 Job Lifecycle

All job state is persisted in SQLite across the three tables above. Figure 5 shows the job finite-state machine: a job progresses *pending* → *searching* → *synthesising* → *reporting* → *evaluating* → *completed*, or transitions to *failed* on any unhandled exception.

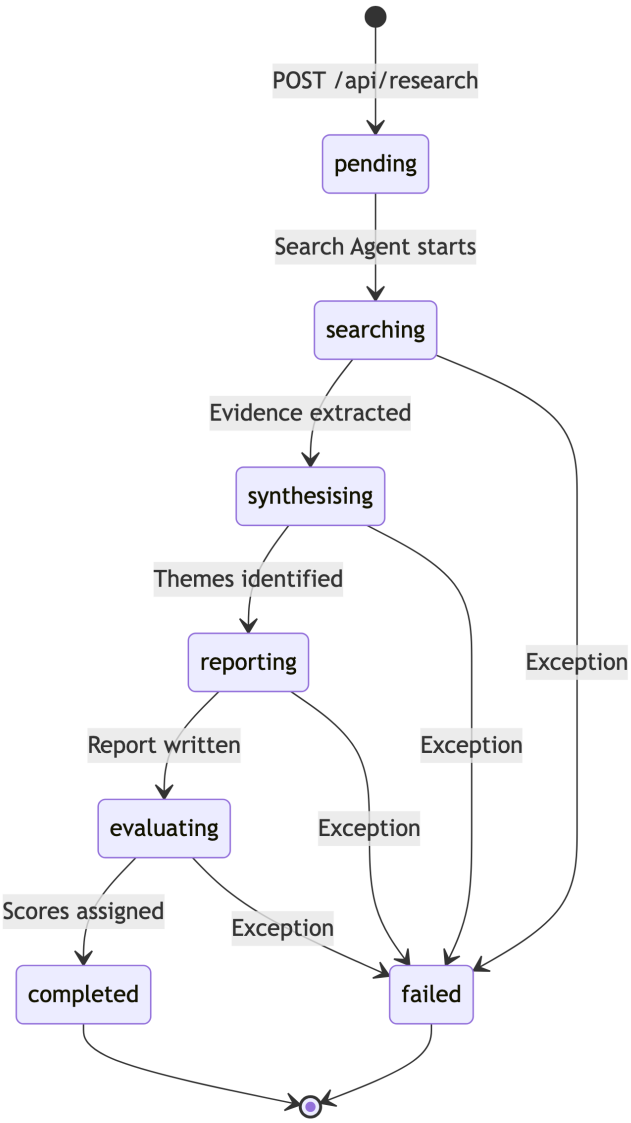

Figure 5: Job status state machine.

4.3 REST API

All endpoints except `/api/health` require `X-API-Key` when `API_KEY` is set. Key endpoints:

Method + Endpoint	Description
`POST /api/research`	Submit question (202)
`GET /api/research/{id}`	Status, report, scores
`GET .../{id}/events`	SSE live stream
`GET .../{id}/pdf`	Download PDF
`DELETE /api/research`	Delete all jobs
`GET /api/health`	Health check

5 Frontend

The Next.js 16 frontend (React 19, Tailwind CSS v4, TypeScript) provides three pages: **Home** (submit question, live status, rendered report), **History** (all jobs with delete), and **Job Detail** (full report with Markdown renderer and PDF download). A `useJobPoller` hook polls the REST API every 2 s and stops on terminal state.

6 Technology Stack

Layer	Technology	Role
Backend	Python 3.11+	Agent & API logic
API framework	FastAPI ≥0.115	HTTP, routing, SSE
ASGI server	Uvicorn ≥0.30	Production server
LLM primary	Groq SDK	llama-3.3-70b
LLM fallback	HuggingFace Hub	Llama-3.3-70B
Web search	ddgs / tavily	DuckDuckGo / premium
Scraping	httpx + bs4	Fetch & parse pages
Validation	Pydantic ≥2	Typed schemas
Database	SQLite (WAL)	Persistent job store
PDF export	WeasyPrint ≥62	Markdown → PDF
Frontend	Next.js 16.2	React 19 App Router
Styling	Tailwind CSS v4	Utility-first CSS
Language (FE)	TypeScript	Type safety
Hosting (FE)	Vercel	Frontend deployment

7 Design Decisions & Security

Decision	Rationale
Sequential agents	Each stage depends on the previous stage's typed output.
SQLite + WAL	Zero operational overhead for single-process demo; clear migration path to PostgreSQL.
`threading`	Avoids Redis/Celery dependency; sufficient for sequential jobs.
Temperature 0.3	Deterministic outputs ease debugging and evaluation.
Dual LLM provider	Groq (speed) + HuggingFace (fallback) improves availability.
API key auth	Header `X-API-Key`; disabled in dev when unset.
CORS allowlist	Only the configured frontend origin is permitted.
Prompt injection	No sanitisation layer currently; a production deployment should wrap user input.

8 Deployment

Figure 6 contrasts the current single-instance setup (FastAPI + SQLite + threads) with a production-scale architecture (load balancer, Celery workers, PostgreSQL). The frontend is deployed on Vercel; the backend runs via `python main.py serve` (Uvicorn). Table 2 lists the key environment variables.

Table 2: Key environment variables (`api/.env`).

Variable	Default	Purpose
`LLM_PROVIDER`	auto	groq / hf / auto
`GROQ_API_KEY`	—	Primary Groq key
`GROQ_API_KEY_2`	—	Fallback Groq key
`HF_TOKEN`	—	HuggingFace token
`TEMPERATURE`	0.3	LLM temperature
`TAVILY_API_KEY`	—	Premium search
`MAX_SEARCH_QUERIES`	3	Queries / question
`MAX_RESULTS_PER_QUERY`	5	Results / query
`MAX_CONTENT_LENGTH`	3000	Chars / page
`API_KEY`	—	Auth key
`CORS_ORIGINS`	Vercel URL	Allowed origins

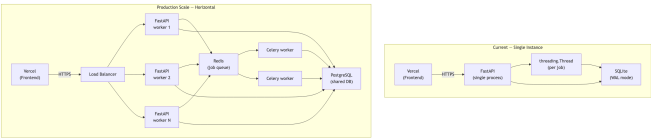

Figure 6: Current vs. scalable deployment.

9 Limitations & Future Work

9.1 Current Limitations

As a demo prototype, ML-ESS makes several pragmatic trade-offs that constrain its use beyond supervised settings:

- **No cross-job memory.** Each research job is independent; the system cannot reuse evidence gathered in a previous run on a related question. A persistent vector store would be needed to enable incremental research across sessions.
- **Self-graded evaluation.** The Evaluator uses the same LLM that produced the report, introducing circular bias. A separate judge model or human annotation loop would improve reliability.
- **Synchronous scraping.** Pages are fetched one at a time with `httpx`; a slow server can add 20–30 s to the Search Agent phase. An async fetch with `asyncio.gather` would cut this by 3×.
- **No source credibility verification.** Credibility notes are LLM-generated heuristics — not checked against domain authority, publication date, or peer-review status.
- **SQLite concurrency ceiling.** WAL mode allows multiple concurrent readers but only one writer. Under sustained load (many simultaneous jobs), write operations queue up.
- **No prompt injection protection.** User questions are passed verbatim to LLM prompts. A production deployment should wrap input in a trusted envelope.

9.2 Planned Extensions

- Streaming LLM output (token-by-token SSE) to reduce perceived latency.
- Async parallel scraping with `asyncio.gather`.
- Cross-job knowledge base via ChromaDB or pgvector.
- Independent evaluator model for unbiased quality scoring.
- Human-in-the-loop evidence annotation before report generation.
- Multi-language output via a dedicated translation agent.
- Horizontal scaling path: Redis job queue, Celery workers, PostgreSQL.

10 Conclusion

ML-ESS demonstrates that a small, well-structured multi-agent system can automate the full research pipeline — from a natural language question to a cited, evaluated report — in minutes. As a demo prototype it prioritises transparency over scale: all intermediate artefacts are persisted and inspectable, and the pipeline can be traced end-to-end without hidden state. Its modular design is its central principle: *a correct, auditable core that grows incrementally.* Inserting a new agent requires adding one function and extending `SharedState`; the REST API and frontend adapt automatically.

Acknowledgements

The authors thank the AIMS Research Innovation Centre and the organisers of the 2026 Doctoral Training School for the research environment and computational resources.

References

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, vol. 33, 2020, pp. 9459–9474.

[2] T. Significant Gravitas, *AutoGPT*, GitHub, 2023. https://github.com/Significant-Gravitas/AutoGPT

[3] J. Moura, *CrewAI*, GitHub, 2024. https://github.com/joaomdmoura/crewAI

[4] H. Chase, *LangChain*, GitHub, 2022. https://github.com/langchain-ai/langchain

[5] Meta AI, "The Llama 3 Herd of Models," *arXiv:2407.21783*, 2024.

[6] S. Ramirez, *FastAPI*, 2019. https://fastapi.tiangolo.com